



**Universidad
Gerardo Barrios**

Carrera:

Ingeniería en Sistema y Redes Informáticas.

Asignatura:

Programación Computacional 4

Tema Para Presentar:

Laboratorio 1 Parte 1 Del Computo 3

Docente:

Ing. William Alexis Montes Girón

Presentado por:

Rene Oswaldo Orellana de la O SMSS018422

Allisson Lourdes Guevara Palma SMIS038421

Introducción

En el desarrollo de aplicaciones web modernas, la eficiencia en la gestión de recursos y la optimización de la experiencia de usuario (UX) constituyen pilares fundamentales para el éxito del software. El proyecto **SkyCast** se presenta como una plataforma modular de monitoreo meteorológico en tiempo real, diseñada estratégicamente bajo una arquitectura desacoplada que integra el robusto ecosistema de **Laravel** en el backend y una interfaz dinámica e intuitiva estilizada con **Tailwind CSS**.

El núcleo técnico de la aplicación no se limita a la mera intermediación de datos externos mediante el consumo de APIs; por el contrario, implementa una capa de persistencia local optimizada a través de un motor de base de datos **SQLite** enfocado en el rendimiento y la consistencia de los datos. Este diseño arquitectónico obedece a la necesidad de mitigar la latencia en las consultas recurrentes y controlar de manera estricta los límites de peticiones del servicio meteorológico, operando como un sistema inteligente de caché y registro histórico de búsquedas.

En la capa de presentación, **SkyCast** traduce la complejidad de las variables ambientales en micro-interacciones limpias y componentes visuales de alta legibilidad. A través de una jerarquía de diseño orientada al usuario, la interfaz procesa asíncronamente los estados de carga y expone módulos críticos como historiales de consulta rápidos y paneles de analítica diaria (Índice UV y Calidad del Aire). El resultado es una solución de software integral, escalable y de alta fidelidad, donde la lógica de negocio y la experiencia visual convergen para ofrecer un servicio fluido, óptimo y de nivel profesional.

Introducción	1
2. Especificación del Modelo de Base de Datos (SQLite)	4
1. Funciones Principales de la Base de Datos	7
2. ¿Por qué está diseñada así? (Arquitectura y Decisiones)	7
3. Funcionalidades de la Capa Visual (Frontend)	8
4. ¿Por qué está diseñada así? (UX y Arquitectura)	8
Reducción de la Carga Cognitiva	9
Optimización del Flujo de Navegación (Micro-interacciones)	9

1. Cuadro de Módulos de la Aplicación (Priorización)

Este cuadro clasifica los componentes de **SkyCast** según su nivel de prioridad (**Alta, Media, Baja**), su funcionalidad y el porqué de su relevancia en la arquitectura

Módulo	Componente Visual / Funcional	Prioridad	Justificación Técnica
Núcleo de Consultas	Buscador Principal e Integración con la API (OpenWeatherMap)	ALTA	Es la funcionalidad central del sistema. Sin la conexión asíncrona para recuperar el clima, la aplicación pierde su propósito operativo.
Persistencia Local	Motor de Caché y Almacenamiento (SQLite)	ALTA	Crucial para el rendimiento. Evita peticiones redundantes a la API externa, disminuye la latencia de carga y estructura los datos en el backend.
Controlador de Datos	Lógica de Negocio y Enrutamiento (Laravel)	ALTA	Administra la comunicación entre el cliente (Vite/Tailwind) y la base de datos, procesando las solicitudes y evitando excepciones críticas de ejecución.
Panel de Métricas	Detalles del Día (Índice UV / Calidad del Aire)	MEDIA	Enriquece la experiencia del usuario agregando valor analítico a la consulta básica de temperatura, distribuyendo la información de forma modular.
Historial Dinámico	Sección de Búsquedas	MEDIA	Permite la reutilización de datos locales en un solo clic, mejorando la

	Recientes en Interfaz		UX y acelerando el tiempo de respuesta visual.
Fidelización y Alertas	Formulario de Suscripción / Notificaciones	BAJA	Es una característica de escalabilidad. Su ausencia no interrumpe el flujo principal de consulta climática actual, pero prepara el sistema para futuras campañas de correo.

2. Especificación del Modelo de Base de Datos (SQLite)

Considerando un diseño relacional estándar para este tipo de arquitecturas en Laravel, aquí se detalla cómo se estructuran los campos (especificando si son VARCHAR, INTEGER, etc.), sus relaciones y el porqué de cada decisión de diseño

Campo	Tipo de Dato	Función / Restricción	¿Por qué se diseñó así?
id	INTEGER	Llave Primaria (Autoincrementable)	Indexación rápida para búsquedas internas en SQLite.
nombre_ciudad	VARCHAR(100)	Nombre de la ubicación consultada	Se define VARCHAR porque las cadenas de texto de nombres de ciudades varían en longitud pero rara vez superan los 100 caracteres.
codigo_pais	VARCHAR(3)	Código ISO del país (ej. "SV")	Longitud fija o corta para búsquedas y filtros regionales precisos.

temperatura	DECIMAL(4,2)	Registro numérico del clima	Permite almacenar valores exactos con decimales (ej. 32.50 °C).
datos_json	TEXT	Payload completo de la respuesta de la API	Guarda toda la estructura secundaria (humedad, viento, índice UV) en un solo bloque de texto plano para no sobrepoblar la tabla con columnas innecesarias.
updated_at	TIMESTAMP	Fecha y hora de la última actualización	Esencial para la lógica de caché; el backend compara este campo para saber si el registro tiene más de 15 minutos de antigüedad y necesita renovarse.

Campo	Tipo de Dato	Función / Restricción	¿Por qué se diseñó así?
id	INTEGER	Llave Primaria	Identificador único de cada evento de búsqueda.
ciudad_clima_id	INTEGER	Llave Foránea (Foreign Key)	Vincula directamente la búsqueda con un registro real en la tabla ciudades_clima.

ip_usuario	VARCHAR(45)	Identificador anónimo del cliente	Almacena la dirección IP (compatible con IPv4 e IPv6) para mostrarle a cada dispositivo únicamente sus propias búsquedas sin obligarlo a registrarse.
created_at	TIMESTAMP	Marca de tiempo de la consulta	Permite ordenar el historial de forma descendente (DESC), mostrando siempre las consultas más recientes primero.

1. Funciones Principales de la Base de Datos

Caché de Consultas (Rendimiento): Almacena las búsquedas recientes de clima por ciudad. Si un usuario vuelve a buscar la misma ciudad en un rango de tiempo corto, el sistema sirve los datos desde la base de datos local en milisegundos, evitando consumir peticiones de la API externa.

Historial de Búsquedas: Registra las ciudades que los usuarios consultan con mayor frecuencia, lo que permite generar estadísticas o sugerencias personalizadas en la interfaz.

Gestión de Favoritos/Alertas: Permite a los usuarios guardar ubicaciones específicas para acceder a ellas directamente al iniciar la aplicación, sin tener que teclear el nombre de nuevo.

2. ¿Por qué está diseñada así? (Arquitectura y Decisiones)

La base de datos sigue un enfoque modular y relacional, estructurada bajo estos pilares:

Optimización de Recursos (API Limits)

Las APIs del clima suelen tener límites de peticiones gratuitas por minuto o por día. El diseño de "tabla espejo" o caché mitiga este problema: antes de disparar un fetch hacia OpenWeatherMap, la aplicación pregunta a la base de datos si ya existe un registro fresco (por ejemplo, menor a 15 o 30 minutos) Si existe, lo usa; si no, actualiza

Integración Limpia con el Backend (JS/Node)

Al utilizar una arquitectura modular en JavaScript, las tablas están normalizadas para que las respuestas del servidor se transformen directamente en objetos JSON limpios. Esto facilita que componentes hechos con Tailwind CSS pinten la información (temperatura, humedad, viento) de forma inmediata y sin transformaciones pesadas en el cliente

3. Funcionalidades de la Capa Visual (Frontend)

Buscador Centralizado ("Cargando..."): Es el punto focal de la aplicación. Su función es recibir la entrada del usuario (ciudad) y disparar la consulta. Mientras recupera los datos (sea de la API o de la base de datos local), muestra un estado de carga dinámico para que el usuario sepa que el sistema está trabajando.

Sección de Detalles del Día (Calidad del Aire / Índice UV): Paneles complementarios que segmentan la información técnica. En lugar de saturar un solo bloque con números, separan variables críticas como la calidad del aire y la radiación ultravioleta.

Módulo de Búsquedas Recientes (Caché Visual): Muestra un acceso rápido a las últimas ciudades consultadas (por ejemplo, San Miguel, SV y San Salvador, SV). Esto se alimenta directamente del historial almacenado en tu base de datos local.

Módulo de Alertas (Fidelización/Suscripción): Un formulario al pie de la página diseñado para capturar el correo electrónico del usuario, preparando el terreno para el envío de notificaciones automáticas o reportes climáticos.

4. ¿Por qué está diseñada así? (UX y Arquitectura)

Jerarquía Visual Limpia (Enfoque en lo Importante) El diseño utiliza un degradado azul que evoca la atmósfera y el clima, manteniendo un contraste alto con contenedores semitransparentes (efecto glassmorphism). Colocar el bloque del clima actual en el centro exacto garantiza que el usuario obtenga la respuesta a su pregunta principal ("¿cómo está el clima aquí?") en menos de un segundo, sin distracciones.

Reducción de la Carga Cognitiva

La información meteorológica puede llegar a ser abrumadora (presión, ráfagas, humedad, puntos de rocío). Al separar los datos en bloques pequeños (Detalles del Día, Búsquedas Recientes), la interfaz se vuelve intuitiva y fácil de leer para cualquier persona, cumpliendo con el principio de diseño modular que manejas en el código de backend.

Optimización del Flujo de Navegación (Micro-interacciones)

El bloque de Búsquedas Recientes no es solo decorativo; está diseñado para evitar clics y escrituras innecesarias. Al hacer clic en una ciudad del historial, el sistema puede disparar una consulta inmediata a la base de datos `database.sqlite` (que ya vimos estructurada en las migraciones de Laravel), trayendo los datos al instante sin volver a llamar a un servicio externo.